

Explainable maintenance decisions with component-level building models and causal knowledge graphs

Project introduction · Technical design · Contributions · Live case study

From one reported duct to traceable root-cause inspection, intervention simulation and occupant-impact explanation.

10,575

MODEL ENTITIES

1,433

KG NODES

4

REPAIR OPTIONS

NUS · BPS5231 AI Application in the Built Environment

Version 3.0 · Building 1 · English edition

Running prototype · Demonstration assumptions disclosed · 27 September 2026

01

Project overview: from occupant observations to explainable maintenance decisions

BuildingCare is a component-level maintenance decision-support prototype for occupants and maintenance teams. Residents report a room, location, description and photographs from a phone. The team locates the component in an interactive 3D model, queries causal knowledge, records inspection evidence one hypothesis at a time, and compares interventions in terms of building services, room use and occupants.

The central question is: why inspect this component, and who would be affected by this repair? Reports, IFC identities, graph evidence, human inspections and scenarios form a traceable workflow. The current system is a running teaching and competition prototype. It demonstrates the decision process while keeping assumptions explicit when field measurements are absent.

Object	Current delivery
Resident interface	Mobile reporting, photo evidence, component suggestions, status and bilingual interim guidance
Maintenance interface	Instance location, hypothesis inspection, visual queries, intervention and impact comparison
Data foundation	10,575 model entities; 71 selectable rooms; 30 GLB chunks
Knowledge foundation	1,433 nodes and 4,959 deduplicated relationships in Neo4j

Demonstration workflow: report → component location → candidate causes → human inspection → intervention selection → dependency animation → cost and occupant-impact charts → report and resident enquiry.

Technical background, problem and research positioning

Occupants describe symptoms such as stuffiness, weak airflow or failed lighting. Building models describe components and spaces, while domain knowledge describes equipment classes, failure modes and potential causes. These representations have different granularity and identities. A model beside a chat window does not by itself establish which knowledge applies to a physical component, or distinguish a hypothesis from confirmation and assumed recovery.

Integration of BIM and building semantic graphs has an established research basis. Brick distinguishes service-flow relationships from spatial location. BIM-to-BRICK examines interoperability between IFC, sensor/BMS information and spatial semantics. This project builds a maintenance-inspection and occupant-impact workflow on these directions; it does not claim to invent BIM, knowledge graphs or GraphRAG. [2][3]

Research question	Prototype response
Which component matches the report?	Room selection, stable identities, text suggestions, explicit binding and source properties
Why was a cause suggested?	Expose failure mode, cause, checks, query parameters and source-row evidence
How do intervention consequences compare?	Save plans, resource inputs, service scope and indicators against one analysis version

The research objective is to reduce fragmented information and the effort of explaining decisions. Current validation establishes executable workflows, identity consistency and rule-calculation correctness; it does not yet demonstrate improved diagnostic accuracy or reduced maintenance cost on real faults.

Architecture and data lifecycle

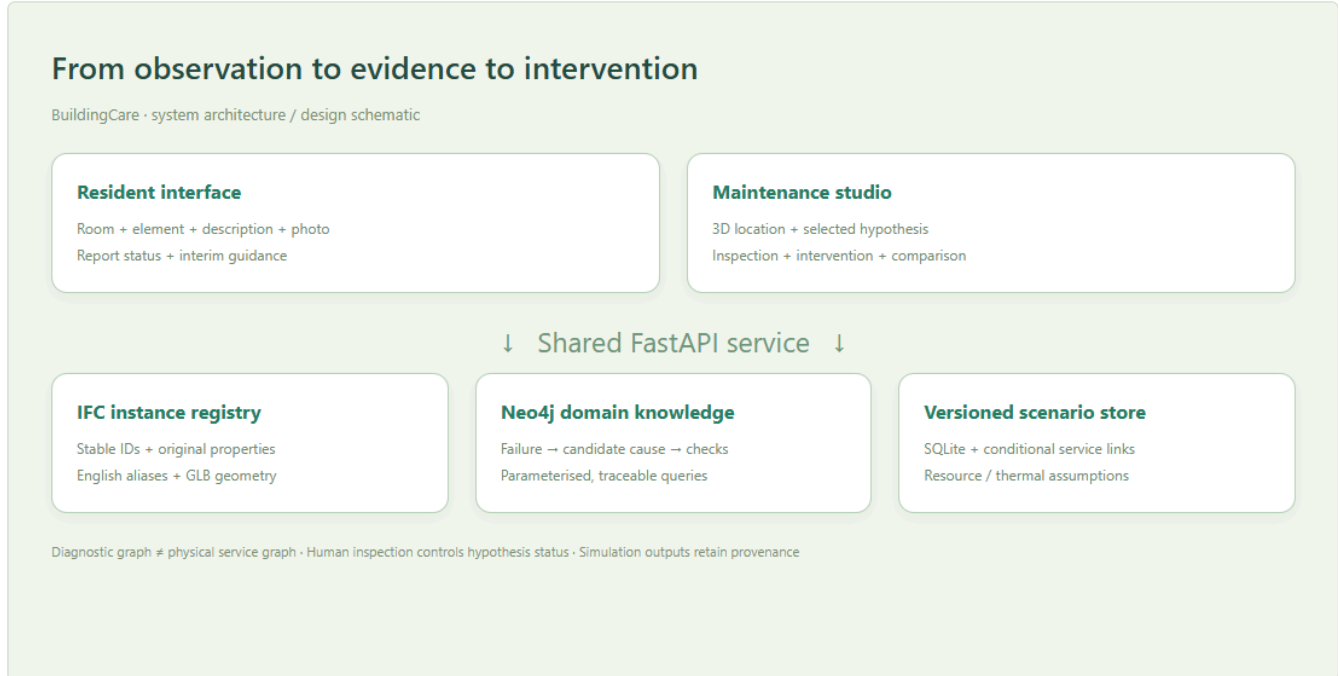


Figure A. System architecture schematic, not a runtime screenshot. Domain knowledge and physical service links have distinct provenance and semantics.

React, TypeScript and Three.js power the maintenance workspace, Investigation Studio, mobile reporting and resident assistant. They share a FastAPI service. SQLite persists reports, identification records, analysis versions, user-entered topology and scenario results; photographs and model chunks remain on the host. Neo4j executes fixed, parameterised Cypher queries against the imported dataset. [1]

An analysis stores a report-location snapshot, dataset hash, graph evidence and topology snapshot. A scenario records analysisId, causeId, intervention method, all inputs, outputs and sources. Changing location, binding or inspection state requires a new analysis version. Older results remain stored but cannot be used as the result of the current report version.

The delivered instance uses local Neo4j; the backend connection boundary can also target a configured remote Neo4j service. Browsers do not receive database credentials or execute arbitrary user-entered Cypher. No public deployment has been completed. Production multi-user hosting still requires authentication, authorisation, TLS, backups and operational logging.

04

Component identity, English naming and location confidence

The display name is Building 1. Component names combine the IFC class, translated source service, original service tag, level and stable short identifier; for example, Supply air · Duct segment · T3 · B1-L02-05050D95. Rooms preferentially use Pset_SpaceCommon.Reference. Room AS-1.01 is not renamed Bedroom without evidence. Original names, floor keys, IFC GlobalIds, coordinates, bounds and properties are retained.

Field	Purpose and boundary
id / ifcGlobalId	Model identity joining browser objects, reports and analyses
name / originalName	Readable English alias / preserved original model name
floorId / floorName	Stable key 02_Kerros / display label Level 02
serviceTag	Original tag such as T3; equal tags do not establish connectivity or room service
roomId / assetId	Reported location, not a confirmed root-cause location

The naming audit covers 10,575 entities and 10,575 distinct display names. Automated comparisons confirm preserved IFC identifiers, coordinates and source properties. The full mapping is model-name-register.csv, with checks in naming-audit.json. This verifies that presentation changes did not alter identities or positions; it does not certify as-built locations or connectivity.

Text-based location suggestions currently use bilingual terms, component classes, source tags and spatial distance, followed by human confirmation. Photo upload and display work, and a vision-service adapter exists. No vision model was configured in this run; arbitrary-photo localisation to an exact duct segment is not claimed.

How the causal graph and query-based reasoning work

The supplied Neo4j CSV export has n, r and m columns. Import auditing yields 5,000 rows, 1,433 nodes and 4,959 deduplicated relationships, including Equipment, FailureMode, RootCause and DiagnosticCheck. In this snapshot, 431 failure modes lack cause edges and 432 lack check edges. Exactly 5,000 rows may reflect an export limit; an unrestricted export is needed to establish coverage.

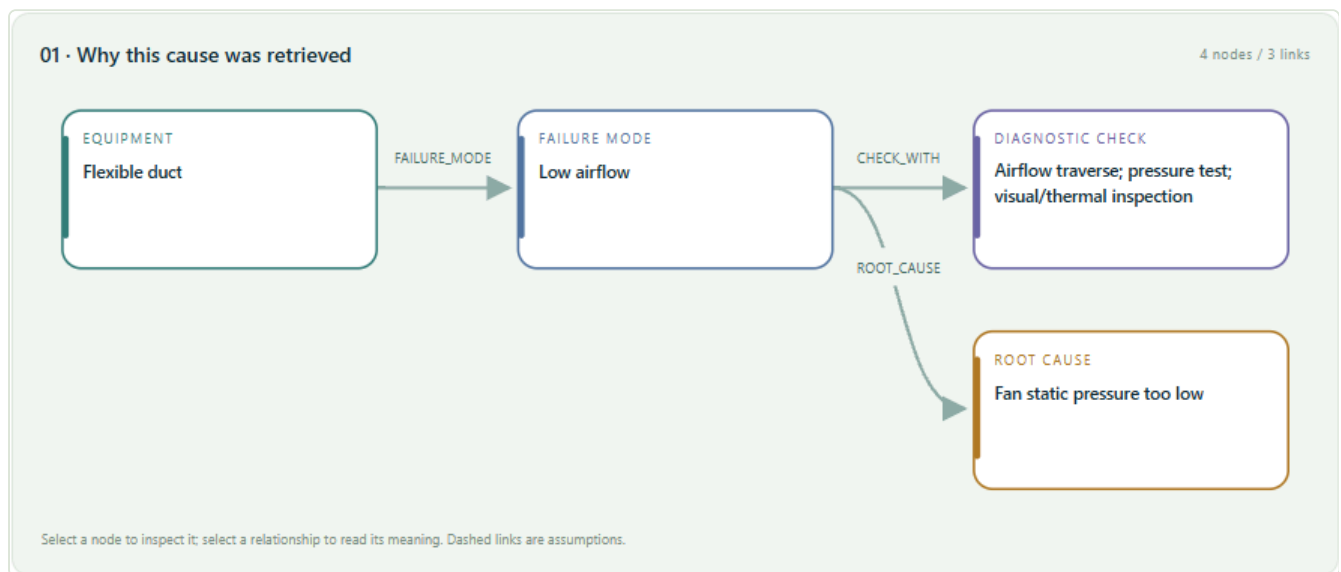


Figure 1. Runtime subgraph from Neo4j querying: equipment class, failure mode, candidate cause and diagnostic check.

The pipeline matches bilingual symptoms, adjusts applicability using model classes and source service tags, selects failure modes, and queries HAS_ROOT_CAUSE and CHECK_WITH using parameterised Cypher. Candidates and evidence are shown separately. HAS_ROOT_CAUSE points from failure mode to candidate cause: a diagnostic lookup direction, not physical failure propagation. The interface exposes the actual query, parameters, returned row count and dataset hash.

This version uses graph-query-supported diagnostic hypothesis retrieval, not probabilistic causal discovery, automatic proof or Neo4j GDS algorithms. Lexical scores rank applicability, not fault probabilities. Equipment classes in the KG still need association with actual IFC instances. Unmapped cases remain uncertain and can be corrected by a person.

Hypothesis inspection and graph-to-model interaction

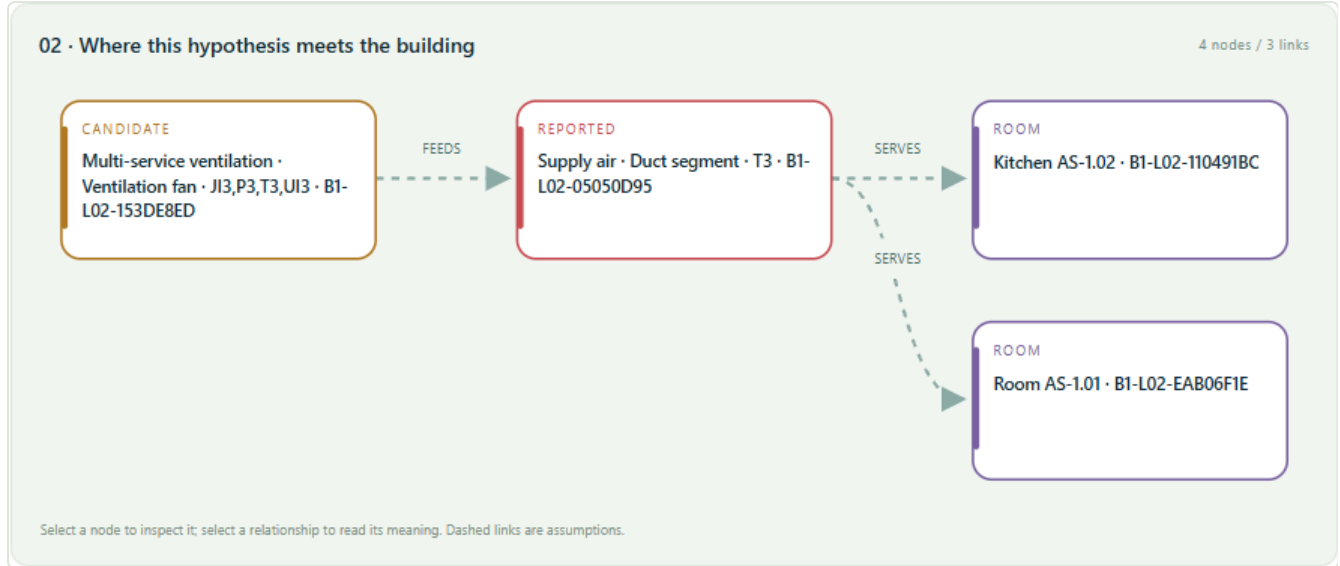


Figure 2. Instance path from the selected upstream-fan hypothesis to the reported duct and rooms; dashed links denote demonstration assumptions.

Reported locations are red, the selected candidate location is amber, and scenario impacts are purple. The recovery frame shows conditional restoration in green. Each hypothesis has its own evidence and instance trace; changing it changes the graph and highlights rather than marking every possible cause red. Nodes locate components, and edges expose relationship meaning, propagation conditions and provenance.

Inspection states are untested, investigating, ruled out and inspector-confirmed. Saving requires an observation or evidence reference, and confirmation requires a component association. Ruled-out hypotheses cannot be simulated. Changing a bound component resets its confirmation. Inspections create a new analysis version and require current comparisons to be recalculated, while retaining history. Confirmation records a human assertion, not automatic validation of a physical fact.

Physical paths come only from entered service links or links isolated within this case. Proximity, a shared floor or a common equipment class never creates SERVES or FEEDS automatically. Missing paths between a candidate and report are explicitly shown.

Repair options, resources and propagation calculations

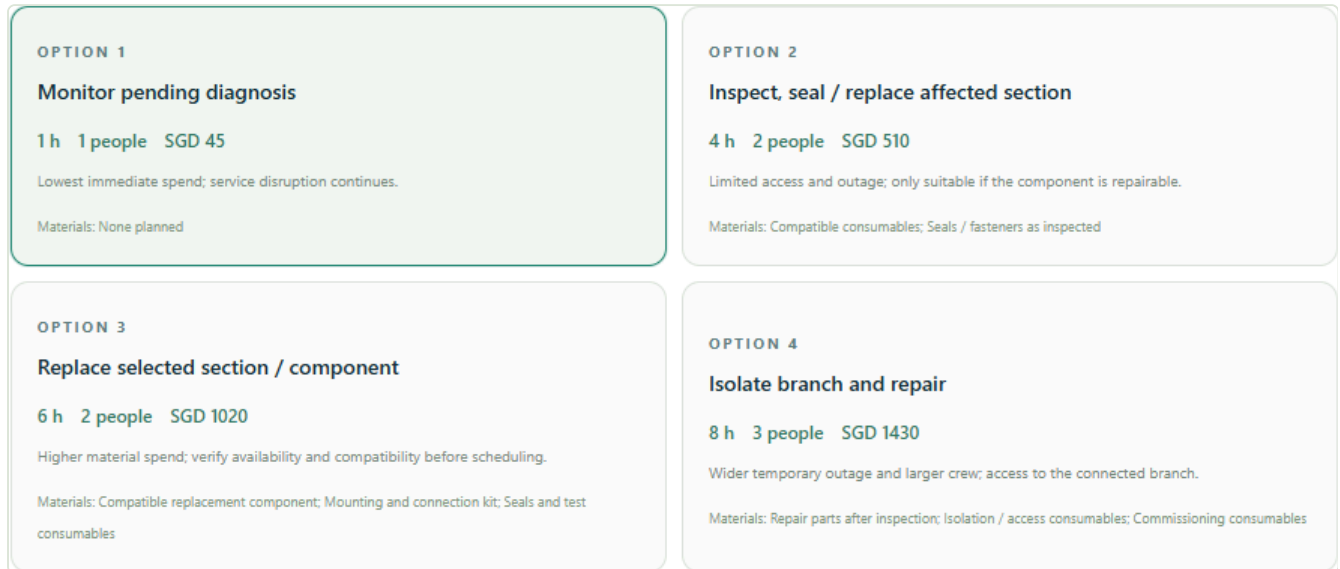


Figure 3. Four intervention cards in the live interface. Time, crew, materials and cost are editable assumptions.

The interface compares monitoring, local repair, component replacement and branch-isolation repair. Options include steps, intended outcomes, material categories and trade-offs. Materials are planning descriptions rather than a live procurement catalogue. Templates vary with the cause, such as filter work or sealing. Chosen duration, crew, hourly rate, material cost, cost range and isolation point are saved with the scenario.

Cost = crew × work hours × hourly rate + material cost. The entered ± percentage is a sensitivity range, not a statistical confidence interval. Taxes, parts lead times, lost business and overhead are excluded. For monitoring, work duration means inspection labour while service interruption lasts over the full planning horizon.

Effects propagate through conditional service links. For example, power loss can become fan ventilation loss through SUPPLIES_POWER, then reach ducts and rooms through FEEDS and SERVES. Traversal state is the pair (entity, service state), and visited states are not expanded twice, preventing infinite cycles. An isolation node must reach the selected cause through those same conditions; unconfigured connections remain unknown.

Occupant and building-performance indicators

Simulation focuses on differences in service scope and resources. Indicators include affected rooms and elements, assumed occupants, occupant-hours of service loss, room-closure hours, service availability over the planning horizon, and air-temperature trajectories and degree-hours when loss of cooling is explicitly assumed. Occupancy is a fixed per-room assumption, not measured presence, productivity loss or health impact.

Indicator	Calculation and interpretation
Occupant service-loss hours	affected rooms × assumed occupants/room × min(outage, horizon)
Room-closure hours	affected rooms × min(outage, horizon), only when closure is selected
Service availability	$100 \times [1 - \text{min}(\text{outage, horizon}) / \text{horizon}]$
Thermal model	$C \, dT/dt = UA(T_{\text{out}} - T) + Q$
Recovery model	$T = T_{\text{initial}} + (T_{\text{repair}} - T_{\text{initial}}) \exp[-(t - t_{\text{repair}})/\tau_{\text{recovery}}]$
Temperature degree-hours	$\int \max(T_{\text{air}} - T_{\text{threshold}}, 0) \, dt$; per-room planning indicator

The thermal model is single-node with constant outdoor temperature and internal gains, using shared inputs for affected rooms. It omits inter-room transfer, humidity, airspeed, solar gains, detailed controls and occupant differences. It is not EnergyPlus, CFD or a calibrated digital twin. Air temperature is not operative or perceived temperature; PMV/PPD, comfort-standard compliance, energy savings and prediction probabilities are not output. Ventilation failure does not automatically imply cooling failure.

Animation reveals elements and rooms by service-dependency hops, followed by conditional recovery. Animation steps do not represent physical propagation speed; repair duration and thermal evolution use their own time quantities. This makes the trace understandable without implying unsupported physical precision.

Demo case study: model objects and scenario

The screenshot displays the BuildingCare workspace interface. At the top, the 'buildingcare' logo and 'BUILDING INTELLIGENCE WORKSPACE' are visible, along with a 'Maintenance desk' link and a 'Resident assistant' button. The main header indicates 'BUILDING 1 / INVESTIGATION STUDIO' and 'Low airflow · Level 02 supply duct'. Below this, the room is identified as 'Room AS-1.01 · B1-L02-EAB06F1E' with a note 'ILLUSTRATIVE CASE'.

The interface is divided into two main sections. On the left, the 'INTERACTIVE BUILDING MODEL' shows a 3D view of 'Building 1' with a 'Level 02' dropdown. A 'SELECTED COMPONENT' panel highlights 'Supply air - Duct segment - T3 · B1-L02-05050D95'. On the right, the 'DIAGNOSTIC GRAPH SEARCH' section features a 'One hypothesis at a time.' heading and a list of hypotheses (H01-H08) such as 'Failed duct sealant/tape', 'Undersized duct/grille', and 'Access panel not sealed'. A 'SELECTED HYPOTHESIS' section details 'Failed duct sealant/tape' with a graph link to 'Air leakage'. At the bottom right, a flowchart titled '01 · Why this cause was retrieved' shows the path from 'EQUIPMENT Supply ductwork' to 'FAILURE MODE Air leakage' and finally to 'DIAGNOSTIC CHECK Airflow traverse; pressure test; visual/thermal inspection'.

Figure 4. Live Building 1 workspace. The reported object is an individual T3 supply-air duct from the model; the defect report is demonstration data.

The case is on Level 02. Room AS-1.01 is reported, with duct asset_05050d9591661e63, fan asset_153de8eda3c22936 and board asset_3313a248aec6c28f. All are real independent entities in the source model. For demonstration, the fan is assumed to supply two branches: the first serves Room AS-1.01 and Kitchen AS-1.02; the second serves Room AS-1.06. Room AS-1.05 is an intentionally disconnected control.

These service links are stored only on this case report, not in the topology shared by ordinary reports. The T3 tag supports source-system identity but does not prove these room-service relationships. No field evidence underlies the case inspections, temperatures, occupancy or costs; the demonstration validates workflow and calculations.

The run and screenshots were captured at 2026-09-27T13:50:34.175188+00:00. The report, current analysis, all four scenario inputs/outputs and browser checks are preserved in evidence/case-run.json for reproduction.

Case steps: from candidate causes to inspection outcome

Step 1: choose Open case study from the maintenance homepage and enter Investigation Studio. The symptom is low airflow at the selected supply duct. Find possible causes or Refresh graph search executes real Neo4j queries and produces candidates. Step 2: select Fan static pressure too low, inspect the path between fan, reported duct and rooms, and verify the assumption labels on dashed edges.

Step 3: enter a DEMO observation assuming normal fan output and mark that cause Ruled out. The interface prevents simulation of that excluded cause. Step 4: select Duct leakage/disconnection, enter a DEMO finding assuming a leaking joint on the chosen T3 duct, and choose Confirmed by inspector. A new analysis version is saved. This is a simulated inspection outcome, not proof of a real leak.

Inspect, then update the hypothesis CONFIRMED

1. Airflow traverse; pressure test; visual/thermal inspection

Suggested type/proximity match only; inspect or explicitly bind a model element.

► Associate a different model component

Inspection evidence / observation

What was checked, the result, and evidence reference. Mark invented case observations as DEMO.

Conclusion

Confirmed by inspector ▼

✓ Save inspection

Latest record: DEMO only; assume an inspection finds a leaking joint on the selected T3 duct. Simulated inspector confirmation; not evidence of an actual building defect.

Saving creates a new analysis version. Previous scenario records remain stored; comparisons must be recalculated for the new evidence.

Figure 5. Saved inspection status and evidence text. The confirmation is explicitly marked as a demonstration, not field inspection.

Step 5: enter Repair & simulate, select each of the four options, and explicitly assume cooling is unavailable in affected rooms. For branch isolation, choose the upstream fan. Each Run selected scenario saves an output, then Compare outcomes compares scenarios belonging to the same analysis version.

Case results: scope, animation and repair graph

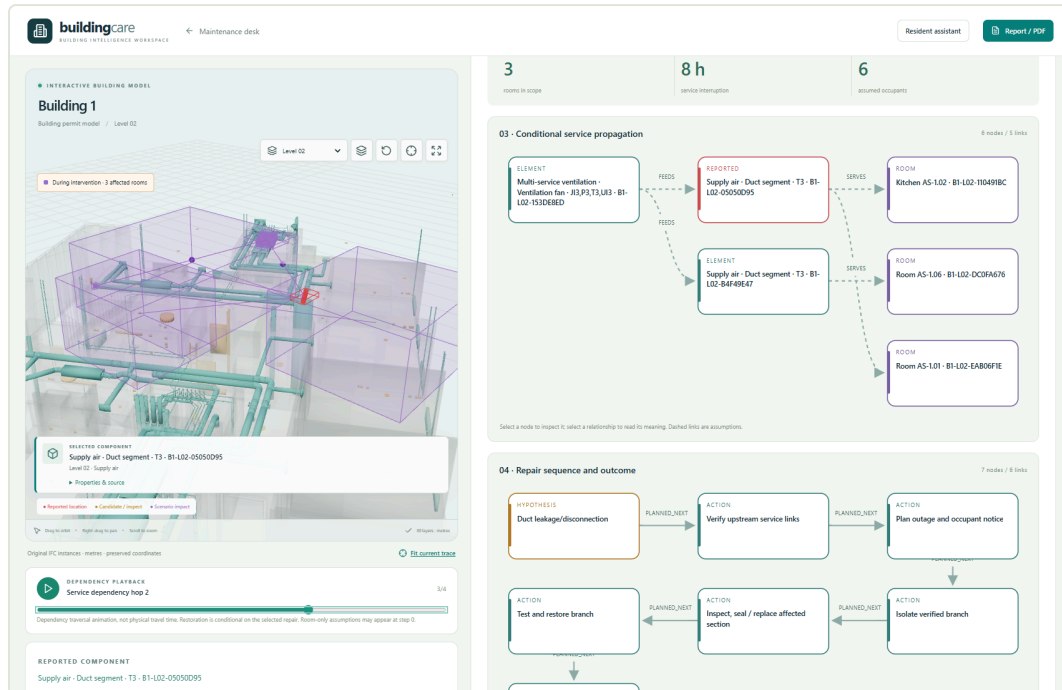


Figure 6. Live model and dependency-playback frame for branch isolation; not a pre-rendered mock-up.

Local repair and replacement start at the selected duct and affect its two assumed served rooms. Branch isolation starts at the fan, expanding scope to two ducts and three rooms. The fourth room has no service edge and is excluded. The rule reaches only configured dependencies rather than marking the entire floor affected.



Figure 7. Intervention sequence leading to conditional restoration. A zero-impact recovery frame assumes correct diagnosis, successful repair and recommissioning.

Case results: comparing cost, crew and occupant impact

Option	SGD	Crew / work h	Rooms	Outage h	Occupant•h	Peak °C
Monitor	45.0	1 / 1	2	24	96.0	31.0
Local repair	510.0	2 / 4	2	4	16.0	25.8
Replace component	1020.0	2 / 6	2	6	24.0	26.6
Branch isolation	1430.0	3 / 8	3	8	48.0	27.3

Shared thermal inputs: initial air temperature 24°C, outdoor 32°C, internal gains 300 W, UA = 150 W/K, capacitance 3 kWh/K, recovery constant 2 h, horizon 24 h and user threshold 28°C. Two occupants per room, a labour rate of 45 SGD/person-hour and a $\pm 25\%$ cost range are assumed. These illustrative warm-weather boundaries are not the source building's climate, NUS measurements or contractor quotations.

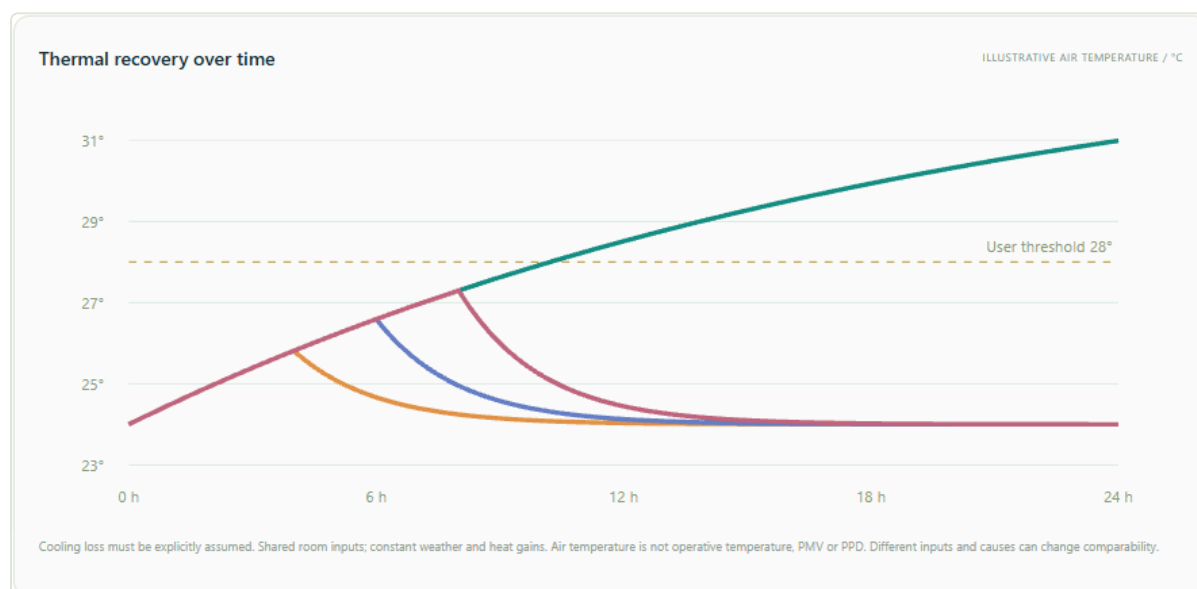


Figure 8. Temperature comparison generated by the live run. Repair options enter assumed recovery after their respective durations, while the monitoring trajectory continues warming.

Under these inputs, local repair has lower estimated spending and occupant service loss than branch isolation. This does not establish its engineering feasibility or replace diagnosis, isolation authorisation and site checks. The platform exposes trade-offs without claiming an unverified optimal solution. Replacement compatibility and lead time still require evidence.

Resident enquiry, interim guidance and reporting

Residents can open Resident assistant directly or after submitting a report. English and Chinese questions receive preset interim guidance. A linked report adds its status and the latest scenario's notice draft for that room. For stuffiness, guidance suggests checking whether belongings block vents, recording time and symptoms, moving to a more comfortable area when necessary, and leaving equipment work to maintenance. It does not instruct residents to dismantle fans, work on live equipment or repeatedly reset breakers.



Figure 9. Real resident enquiry at a phone viewport. The interface labels preset guidance, no LLM call, and the draft status of the maintenance notice.

The existing maintenance LLM interface remains available for configuration, but neither a real LLM nor a vision service was configured in this run. The usable resident demonstration is deterministic guidance plus saved report data, not validated live-LLM conversation. Reports support print/PDF and cite graph, inspection and scenario evidence; evidence/live-maintenance-report.pdf is exported from this run.

Technical contributions and disclosure-style description

Technical field: building facilities maintenance, building information modelling, knowledge-graph retrieval and repair-scenario decision support. Technical problem: heterogeneous reports, generic failure knowledge and building instances lack auditable correspondence, making cause evidence and repair consequences difficult to explain together. Proposed solution: combine identity-preserving instance registration, provenance-aware cause retrieval, versioned human inspection, state-transforming service-topology traversal and occupant-impact presentation.

Contribution	Implemented technical content	Further validation
Identity-preserving report/model alignment	English aliases preserve IFC identity; individual location and human verification	Resident location time and error rate
Separate diagnosis from physical propagation	Distinct storage and display for causal lookup and conditional service edges	Expert comprehension of path explanations
Evidence-version-constrained comparison	Inspection/location changes invalidate current use of old scenarios while retaining history	Concurrent editing, approval and audit robustness
Occupant-oriented repair consequences	Compare cost, materials, time, scope, service loss and thermal curves	Calibrated performance models, occupant studies and decision quality

Disclosure-style process: S1 register components and spatial references; S2 receive and confirm a reported instance; S3 query paths from failure modes to causes and checks; S4 save inspection state and create a new analysis version; S5 calculate impact using the selected isolation scope and conditional service graph; S6 calculate resource and occupant indicators and produce corresponding charts, animation and a traceable report.

These are claims about the project' s technical combination and implementation contributions, not world-first claims, a granted patent or a patentability conclusion. No comprehensive novelty search is included. The recommended competition positioning is an explainable workflow, reproducible experiments and explicit treatment of uncertainty.

Validation, reproduction and next stage

Offline animation: evidence/simulation-playback.gif, made from actual browser frames. The platform supports playback, pause and a step slider.

Validation includes real Neo4j dataset connectivity/querying, all 30 model chunks, individual hypotheses, blocking ruled-out causes, versioned inspector confirmation, four saved options, distinct local/branch scopes, animation and recovery, saved results after reload, mobile enquiry, no page overflow, and a report PDF. No page errors or server 5xx responses were recorded. Automated tests additionally cover cycles, propagation conditions, location changes, uploads and persistence. See VALIDATION_V3.md.

Running now	Not yet validated or implemented
Instance model and English registry	Complete field verification and as-built topology
Neo4j query and evidence paths	Full KG coverage, real-fault accuracy and probabilistic inference
Dependency propagation and resource estimates	Engineering airflow/thermal-moisture simulation; calibrated durations and quotes
Deterministic bilingual resident guidance	Configured and accepted LLM/vision services; production access controls

Reproduce: run `.\restart.ps1 -Build from maintenance-platform`, open `http://localhost:8000/` and choose Open case study. Resident routes are `/report` and `/resident`. The local Neo4j browser is `http://127.0.0.1:17474/`. Credentials remain in the local `.env` and are excluded from the dossier. Photographs, model data and database are in data. The complete case-replay script is `tools/capture_case_v3.py`.

Next, complete and review the KG and instance service topology, then collect real reports and expert labels as a test set. Configure LLM/vision services and evaluate localisation/retrieval before calibrating performance models with temperature, humidity, airflow or BMS data. Suitable measures include top-k cause coverage, localisation accuracy, false impact scope, task time and prediction error. No unmeasured values are filled in here.

References: [1] Neo4j Python Driver Manual, Query the database (parameterised queries); [2] Brick Ontology, Relationships (location and service relationships); [3] BIM-to-BRICK: Using graph modeling for IoT/BMS and spatial semantic data interoperability within digital data models of buildings, arXiv:2307.13197.

<https://neo4j.com/docs/python-manual/current/query-simple/>

<https://docs.brickschema.org/brick/relationships.html>

<https://arxiv.org/abs/2307.13197>